UNIVERSIDAD NACIONAL, SEDE REGIONAL BRUNCA
EIF209 Programación IV
Prof. Daniel Granados Murilo
Prof: Rubén Mora Vargas
Prof: Juan Gamboa Abarca

PROYECTO FINAL
"Gestión del fin"

Objetivo

Involucrar al estudiantado en la solución de un proyecto informático completo de mediana complejidad, mediante el desarrollo de una aplicación web, el diseño de la infraestructura para su operación y la implementación en un ambiente real, para reforzar el aprendizaje y validar los conocimientos adquiridos durante el tiempo del curso.

Aspectos generales

1. Lea y comprenda cuidadosamente lo que se le solicite.
2. El Proyecto puede ser realizado en grupos de 5 personas como máximo, según capacidad del grupo y aceptación del profesor. La idea de que trabajen hasta 5 estudiantes es para disminuir la carga en cuanto a tener que hacer la API igual, el logger, seguridad y conexión con base de datos, y que se centren en lo interesante del proyecto, las soluciones propias y consigan un cumplimiento igual al 100% o muy cercano.
3. El día de entrega, el proyecto será defendido de manera individual. La nota será mérito personal por lo que deben conocer todas las particularidades del sistema y tener pruebas de trabajo realizado en cuanto a documentación y repositorio Github.
4. Se deben respetar las etapas del proyecto mencionadas en el programa del curso:

Se trabaja con 4 etapas.

Base Inicial(8%): Se revisa la arquitectura y diseño del trabajo, diseño de datos y páginas web y la conexión con la API.

Aplicación Base(10%): Se revisan las funcionalidades del sistema final y se proponen cambios para la defensa final, refactorización de código, y seguridad.

Defensa Del Proyecto(14%): Se revisa la implementación del sistema, Estrés de la aplicación con datos, Cumplimiento de requerimientos y pruebas de integración.

Nota: Si el proyecto no tiene un 80% del alcance solicitado, pierde la posibilidad de realizar la presentación final.

Presentación Final(8%): Se corrigen los últimos detalles que el profesor indica, realiza una presentación a los compañeros, profesor e invitados, presenta alcance, conclusiones y recomendaciones del proyecto.

5. Si se comprueba que existen dos o más proyectos similares o copiados, se procederá a colocar nota cero a todos los proyectos involucrados.
6. Se requiere acceso durante toda la revisión al repositorio de GitHub, donde está el proyecto, así mismo debe compartir una carpeta en drive con la copia final del código fuente como respaldo, y los archivos de documentación adicionales. Deben subir al Aula Virtual solo un archivo de texto con los dos enlaces solicitados, ambos enlaces deben apuntar a repositorios compartidos con la cuenta rmoravargas@gmail.com, daniel.granados.dev.566@gmail.com , francisco.gamboa.abarca@una.cr en tiempo y forma.
7. Fechas importantes:

Cronograma del Proyecto – Pérez Zeledón

Hito (Semana)	Fecha
Entrega del enunciado (Semana 2)	23 de febrero de 2026
Base Inicial (Semana 10)	27 de abril de 2026
Aplicación Base (Semana 14)	25 de mayo de 2026
Defensa del Proyecto (Semana 15)	1 de junio de 2026
Presentación Final (Semana 17)	15 de junio de 2026

Cronograma de Hitos del Proyecto – Coto

Hito (Semana)	Fecha
Entrega del enunciado (Semana 2)	25 de febrero de 2026
Base Inicial (Semana 10)	29 de abril de 2026
Aplicación Base (Semana 14)	27 de mayo de 2026
Defensa del Proyecto (Semana 15)	3 de junio de 2026
Presentación Final (Semana 17)	17 de junio de 2026

Contexto del proyecto

En el mundo ha pasado lo que muchos temen, ha llegado un apocalipsis zombie, nadie sabe quien ocasionó este desastre, pero en esta circunstancia sólo queda actuar y organizarse.

Los humanos restantes se han resguardado en pocos campamentos, en donde tienen diversos recursos, como: la comida, el agua, productos de higiene y de defensa, también puede ser munición necesaria para defenderse, además existe más de un campamento, en ocasiones se hacen traslados entre municiones o personas, y se está informando acerca de la situación de otros campamentos, por lo cúal, nuestra tarea es crear un sistema, para gestionar a las personas de los campamentos, así como la llegada de nuevas personas, se debe manejar el ingreso y egreso de recursos en la base, además de poder realizar exploraciones a lugares los cuales pueden contener recursos o ser valiosos en posición.

Requerimientos del sistema.

Requerimiento	Progreso
Gestión humana dentro de la base	
1 - El sistema debe permitir el acceso solo a aquellas personas a las cuales tengan un rol en específico para el manejo de este. No se quieren fugas de información, además que se espera que si el usuario no ha realizado ninguna acción luego de 20 minutos, el sistema se bloquee.	
2 - Para el ingreso de una nueva persona, se debe realizar una toma de información respecto a el contexto, un apocalipsis zombie, (el estudiante debe definir la información y argumentar porque es valiosa), además de poder agregar imágenes y una tarjeta de identificación. Esta información debe ser procesada de forma automática por una IA que permita demostrar cómo se tomó la decisión de manera transparente, en donde en base a ciertas reglas del campamento se permita o no, el ingreso de la persona, se debe generar un reporte de la aceptación o rechazo, para el que usuario final pueda decidir si el análisis es correcto o incorrecto y deje ingresar o no a la persona.	
3 - Cuando se ingresa a una persona, se le debe agregar una identificación, y un cargo o profesión, esto de forma automática con la información dada (los cargos/roles debe definirlos el estudiante) y la IA los debe asignar de acuerdo a reglas específicas y a registros previos.	
El sistema debe gestionar las condiciones de las personas, ya sea que la persona esté enferma, herida, fuera del campamento, entre otros. Según sea su estado no podrá realizar su trabajo, por lo que, si uno de los trabajos/profesiones se queda sin trabajadores, se debe poder mover a personas de otro trabajo/profesión de forma temporal a la que se quedó sin personas.	

Requerimiento	Progreso
Gestión de recursos	
1 - Se debe tener en la bodega, un conteo de todos los recursos existentes, los cuales puedan ser consultados.	
2 - Se deben de generar alertas cuando un recurso esté por debajo de un mínimo establecido.	
3 - Recursos como la comida y el agua deben de conseguirse a diario, estas se reflejan en el trabajo de cada uno de los integrantes del campamento, su trabajo indica cuánta comida o agua consiguen a diario,(se debe crear un pequeño apartado para ingresar un cambio de este ingreso, en caso de que la persona no pueda cumplir con su objetivo) y de igual forma a diario se les da una ración a cada persona, por lo que estos cambios deben de contabilizarse en la bodega principal, estos procesos deben de realizarse de forma automática y entender la dinámica de cada campamento.	
4 - El sistema deberá poder gestionar exploraciones, las cuales hacen que un grupo de personas con ese trabajo, puedan salir a buscar raciones, estas exploraciones consumen raciones adicionales, se deben agendar y tener un control de los días que puede tomar la exploración y algunos días extras como último recurso, al llegar al campamento se debe hacer un recuento de las provisiones que han traído, y estas deben de contabilizarse en el sistema.	

Requerimiento	Progreso
Otros campamentos	
1 - El sistema deberá manejar más de un campamento y gestionar la información de manera separada (similar a un sistema multiempresa).	
2 - Los campamentos pueden enviar solicitudes de recursos o ayuda a otro campamento vecino, estas solicitudes pueden aprobarse o rechazarse, en caso de aprobarse, deben de reflejarse los cambios de en la bodega, esto es vital por lo que deben haber información para su auditoría siempre.	
3 - Si se aprueban las solicitudes, en caso de necesitar enviar personas, se debe de crear un grupo con personas que tengan ese rol, además de que deben llevar raciones para su viaje. Y en caso de prestar raciones a otra base, se debe agendar la entrega de dichos recursos, y debe ser aprobada por ambos campamentos, tanto a la hora de salir, como a la hora de ingresar para contabilizar el manejo de la bodega.	

Requerimiento	Progreso
Generales	
1 - Se debe crear un dashboard en el cual se puedan tener métricas importantes sobre datos del campamento y la bodega (Gestor de recursos, y administrador sistema), para los demás, solo mostrar los recursos que se le asignan a cada persona.	
2 - Si se cambia de un campamento a otro, el sistema debe de volver al inicio y permitir sólo acceso a la información autorizada.	
3 - Se debe implementar un diseño, animaciones y estilos conforme al contexto del proyecto, sin dejar de lado las mejores prácticas de UI y UX para programación de aplicaciones web.	
4- Para poder controlar ciertos flujos y asegurar la supervivencia de todos es importante que se maneje una sola hora en el servidor central, ya que este está resguardado contra zombies y otros invasores.	
5 - Roles contemplados. 5.1 - (Administrador sistema) - Tiene acceso al ver todo el sistema, pero solo gestiona los ingresos de personas 5.2 - (Trabajardor) - Solo puede hacer cambios de inventario, autorizados por el gestionador de los recursos. 5.3 - (Gestión recursos) - Encargado general de realizar traslados y envíos de recursos. 5.4 - (Encargado de viajes y comunicación) - Realizada las expediciones y negociaciones con los otros campamentos. Se puede incluir otro rol si el estudiante lo encuentra necesario y justificable.	

Requisitos No Funcionales:

Arquitectura y tecnologías

El sistema debe desarrollarse bajo una arquitectura moderna orientada a servicios, con una separación clara entre la aplicación web y la API. La comunicación entre ambas capas debe realizarse mediante una API versionada y documentada, permitiendo la escalabilidad y el mantenimiento del sistema. El frontend debe implementarse utilizando **React 18 con TypeScript y Vite**,

aplicando una organización por capas o por funcionalidades, así como patrones de diseño apropiados al desarrollo frontend moderno, debidamente justificados por el estudiante.

Criterios de aceptación:

- Existe separación clara entre frontend y backend.
- El proyecto utiliza React 18, TypeScript y Vite.
- La estructura del código es coherente y justificable en la defensa.

Desarrollo asistido por inteligencia artificial

El proyecto debe incorporar de manera explícita el uso de **inteligencia artificial como apoyo al diseño y desarrollo del sistema**. El estudiante debe evidenciar cómo la IA fue utilizada para apoyar decisiones técnicas, generación de código, reglas de negocio o procesamiento de información.
Las decisiones automatizadas relevantes deben ser **explicables**, permitiendo al usuario final comprender los criterios utilizados y validar o corregir dichas decisiones cuando sea necesario.

Criterios de aceptación:

- Se documenta el uso de IA en el desarrollo del proyecto.
- Las decisiones asistidas por IA muestran criterios claros y trazables.
- El usuario puede revisar y aceptar o corregir decisiones automatizadas.

Calidad de código y buenas prácticas

El código fuente debe cumplir estándares profesionales de desarrollo, promoviendo la mantenibilidad y la legibilidad. Se debe integrar el uso de **ESLint, Prettier y CSpell** como parte del flujo de trabajo, así como configuraciones estrictas de TypeScript.
La aplicación debe seguir las buenas prácticas de **React 18**, incluyendo componentes reutilizables, separación de responsabilidades y manejo adecuado del estado y los efectos.

Criterios de aceptación:

- El proyecto pasa por validaciones de linting y formateo.
- El código es consistente, legible y bien organizado.
- Se evidencia el uso correcto de prácticas modernas de React.

Experiencia de usuario, animaciones y gamificación

La aplicación debe ofrecer una experiencia de usuario moderna, coherente con el contexto del proyecto, integrando **animaciones avanzadas y microinteracciones** que refuercen la narrativa y el feedback al usuario. Asimismo, el sistema debe incluir elementos de **gamificación**, como progresos, niveles, logros o recompensas visuales, integrados de forma funcional a los procesos del sistema.

Criterios de aceptación:

- Existen animaciones coherentes y funcionales (cargas, transiciones, acciones).
- La gamificación aporta valor real a la experiencia del usuario.
- La interfaz mantiene coherencia visual y temática.

Rendimiento y adaptabilidad

El sistema debe optimizar el uso de recursos mediante mecanismos como **paginación, carga diferida y manejo eficiente de procesos asíncronos**, evitando sobrecargar la interfaz del usuario.
La aplicación debe ser **responsive**, garantizando una experiencia adecuada tanto en dispositivos móviles como en computadoras de escritorio.

Criterios de aceptación:

- La aplicación gestiona correctamente grandes volúmenes de información.
- El comportamiento es adecuado en conexiones lentas.
- El diseño se adapta correctamente a distintos tamaños de pantalla.

Seguridad, control de acceso y sesión

El sistema debe implementar mecanismos de **autenticación y autorización por roles**, restringiendo el acceso a la información según el perfil del usuario. La sesión del usuario debe expirar tras un período de inactividad, bloqueando el acceso hasta una nueva autenticación.

Criterios de aceptación:

- Existen roles claramente definidos y aplicados.
- El acceso a la información está correctamente restringido.
- La sesión expira tras inactividad.

Consistencia temporal

El sistema debe operar bajo una **hora única centralizada**, utilizada para procesos críticos como consumo de recursos, exploraciones y transferencias entre campamentos, garantizando coherencia temporal en todo el sistema.

Criterios de aceptación:

- Las operaciones dependen de la hora del servidor.
- No existen inconsistencias temporales entre usuarios o campamentos.

Pruebas automáticas

El proyecto debe incluir **pruebas automáticas de extremo a extremo (E2E) utilizando Playwright**, cubriendo los flujos críticos del sistema. Estas pruebas deben ejecutarse de forma automática como parte del proceso de integración continua.

Criterios de aceptación:

- Existen pruebas E2E para los flujos principales.
- Las pruebas pueden ejecutarse y reproducirse correctamente.
- Los errores críticos son detectados antes del despliegue.

Despliegue y operación

El sistema debe publicarse utilizando servicios en la nube en sus capas gratuitas, con el repositorio alojado en **GitHub** y el frontend desplegado en **Vercel**, asegurando que la aplicación sea accesible públicamente.

Criterios de aceptación:

- El repositorio es público y accesible.
- La aplicación se encuentra desplegada y funcional.
- El proceso de despliegue es reproducible.

Criterios de evaluación

Como directriz de la cátedra se establece como obligatoria la defensa del proyecto, por parte de todos los miembros del grupo, la nota será siempre tomada de manera individual.

La presente rúbrica permite evaluar los proyectos basados en los siguientes rubros:

Rubro	Valor
1. Base Inicial Se revisa la arquitectura y diseño del trabajo, diseño de datos y páginas web y la conexión con la API.	**20%**
2. Aplicación Base Se revisan las funcionalidades del sistema final y se proponen cambios para la defensa final, refactorización de código, y seguridad.	**25%**
3. Defensa Del Proyecto Se revisa la implementación del sistema, se realizan pruebas de estrés de la aplicación con muchos datos, se valora el cumplimiento de los requerimientos y se revisan las pruebas de integración y procesos. **Nota: Si la defensa no tiene un 80% del alcance solicitado, pierde la posibilidad de realizar la presentación final, la nota queda con lo que haya sumado.**	**40%**
4. Presentación Final Se corrigen los detalles que el profesor indica, se realiza una	**15%**

presentación a los compañeros, profesor e invitados, presenta alcance, conclusiones y recomendaciones del proyecto.	
Total	**100%**

La escala de puntos indica el rango de puntos en los que puede ser puntuado cada uno de los criterios de la rúbrica según los avances o proyecto presentado por el estudiante.

1. **Base inicial(8 puntos).**

Rubro	Alto (6 a 8)	Suficiente (2 a 5)	Bajo (0 a 1)
Arquitectura y diseño del trabajo.	Realiza un esfuerzo alto por agregar y entender las relaciones de los componentes de la aplicación, lo demuestra en un diagrama de draw.io o similar	Hay falencias en la interpretación de los elementos y las relaciones de los componentes de la aplicación o el diagrama presenta pocos detalles a mejorar.	No realiza el trabajo, es vago en su realización o no cumple con lo solicitado
Diseño de Páginas web	Presenta un diseño completo de las principales pantallas, utiliza metáforas comunes del desarrollo web, demuestra interés y creatividad, el mockup es bastante completo. Conoce todas las	Presenta un diseño bastante bueno, faltan algunos detalles, rutas o páginas importantes, o los diseños son muy simples o enredados.	No demuestra interés en lograr el diseño, o lo realiza vagamente o tiene faltantes más de dos páginas importantes.

	posibles rutas		
Diseño de datos	Presenta un diseño completo de las principales entidades, documentos, atributos y sus relaciones, demuestra interés, análisis y creatividad, el diseño es bastante completo. Conoce todas las posibles rutas	Presenta un diseño bastante bueno, faltan algunos detalles, atributos o relaciones, o los diseños son muy simples o enredados.	No demuestra interés en lograr el diseño, o lo realiza vagamente, o tiene faltantes más de dos entidades importantes.
Conexión con la API.	Se desarrolla la API del sistema, y se consume desde la aplicación web	Solo existe la API, está bien formada, tiene pocos faltantes o presenta algún bug menor.	No existe la API, está incompleta, o no está bien formada
Entrevista sobre diseño,desarrollo y entendimiento del tema	Demuestra dominio técnico del proyecto. Explica con claridad la arquitectura, decisiones de diseño, patrones utilizados, manejo de errores, seguridad y optimización. Justifica sus elecciones técnicas con fundamentos sólidos y evidencia comprensión integral del sistema.	Explica parcialmente el diseño y desarrollo. Conoce la estructura general del proyecto, pero presenta dudas en decisiones técnicas, arquitectura o aspectos específicos de implementación.	No logra explicar adecuadamente el diseño ni las decisiones técnicas. Evidencia falta de comprensión del funcionamiento general, arquitectura o lógica del sistema.

2. Aplicación Base.

Rubro	Alto (6 a 8)	Suficiente (2 a 5)	Bajo (0 a 1)
Funcionamiento integral de la aplicación web	Se desarrolló el aplicativo completo. Puede que le falten detalles menores a mejorar o detalles de diseño u optimización.	Se desarrollaron parte de los elementos/ requerimientos del aplicativo solicitado, pero queda pendiente gran parte del mismo. Lo que se hizo funciona al menos al 50%, o presenta varios errores considerables.	Existe más de un 50% de faltantes en la aplicación, o no funcionan correctamente, o están mal estructuradas, o no tienen control de errores.
API Funcional y datos bien formados	Presenta una API completa, robusta, bien estructurada y funcional, tiene seguridad, requiere pocas refactorizaciones	Presentan una API casi completa con faltantes en optimización o falta manejo completo de errores.	No existe la API, está en estado básico o tiene faltantes para el manejo de entidades o errores.
Entrevista sobre diseño,desarrollo y entendimiento del tema	Demuestra dominio del diseño y desarrollo del proyecto. Explica con claridad las decisiones técnicas, arquitectura, patrones utilizados y manejo de errores. Responde con seguridad, fundamenta sus elecciones y demuestra comprensión integral del problema.	Explica parcialmente el diseño y desarrollo. Tiene nociones generales de la arquitectura y decisiones tomadas, pero presenta dudas o inconsistencias en aspectos técnicos relevantes.	No puede explicar adecuadamente el diseño ni las decisiones técnicas. Evidencia falta de comprensión del funcionamiento general, arquitectura o lógica del proyecto.

3. Defensa Del Proyecto

Rubro	Alto (6 a 8)	Suficiente (2 a 5)	Bajo (0 a 1)
Aspecto visual final	Se desarrolló desde el inicio un web agradable a la vista, en donde se toma en cuenta al usuario, siendo amigable, evitando acciones	Se desarrolló en parte un aspecto visual agradable, con algunas carencias en aspectos de amabilidad de usuario, comodidad visual, usabilidad y el	Existen una gran deficiencia en el diseño, aspectos como amabilidad con el usuario, usabilidad son casi nulos, el diseño entorpece el

	innecesarias mejorando la velocidad de los procesos, tiene un diseño relacionado con la temática y además cada tiene el mismo nivel de detalle	diseño de la web no mejora los procesos	proceso.
Funcionamiento integral de la aplicación web	Se desarrolló el aplicativo completo. Se corrigen los detalles de la evaluación anterior	Se desarrollaron parte de los elementos/ requerimientos del aplicativo solicitado, pero queda pendiente gran parte del mismo. Lo que se hizo funciona al menos al 70%, o presenta varios errores considerables.	Existe más de un 30% de faltantes en la aplicación, o no funcionan correctamente, o están mal estructuradas, o no tienen control de errores.
API Funcional y datos bien formados	Presenta una API completa, robusta, bien estructurada y funcional, tiene seguridad y está refactorizada	Presentan una API casi completa(80%) con faltantes en optimización o falta manejo completo de errores.	No existe la API, está en estado básico o tiene faltantes para el manejo de entidades o errores.
Pruebas de estrés	El sistema cuenta con mecanismos para trabajar con volúmenes de datos altos, o mejorar la carga al usuario, y los estudiantes muestran este funcionamiento según lo sugerido.	El sistema presenta algunas pruebas o estas fallan, se realiza un muy buen intento por parte de los estudiantes.	No hay acciones para mejorar el rendimiento de la aplicación, o su carga, o no se aplican correctamente, o el esfuerzo es insuficiente.
Entrevista sobre diseño,desarrollo y entendimiento del tema	Demuestra dominio técnico del proyecto. Explica con claridad la arquitectura, decisiones de diseño, patrones utilizados, manejo de errores, seguridad y optimización. Justifica sus elecciones técnicas	Explica parcialmente el diseño y desarrollo. Conoce la estructura general del proyecto, pero presenta dudas en decisiones técnicas, arquitectura o aspectos específicos de implementación.	No logra explicar adecuadamente el diseño ni las decisiones técnicas. Evidencia falta de comprensión del funcionamiento general, arquitectura o lógica del sistema.

	con fundamentos sólidos y evidencia comprensión integral del sistema.		

4. Presentación Final

Rubro	Alto (6 a 8)	Suficiente (2 a 5)	Bajo (0 a 1)
Presentación del proyecto	Presentan un proyecto basado en objetivos y soluciones y conclusiones, involucran a los compañeros y atienden todas las dudas señaladas.	Presenta el proyecto de una manera básica, atienden las dudas señaladas o la presentación es aburrida o no involucra al público.	No presentan el proyecto, no está bien estructurada la presentación, o no atienden dudas o no las pueden responder.
Recomendaciones y conclusiones	Presenta una documentación completa del proyecto e incluye recomendaciones y conclusiones importantes.	Presenta la documentación completa y las recomendaciones y conclusiones no son significativas.	No presenta documentación, o no presenta conclusiones o recomendaciones.
Corrigieron todos los detalles señalados en la sesión anterior.	Se corrigen todas las recomendaciones dadas, presentan un código refactorizado y limpio	Solo existe la API, está bien formada, tiene pocos faltantes o presenta algún bug menor.	No existe la API, está incompleta, o no está bien formada